CS673 Software Engineering

Team 1 - BariatricPath

Project Proposal and Planning

Team Member	Role(s)	Signature	Date
Fatimah Hassan	Security lead | configuration lead	*Hassan F.K*	05/11/2026
Kai Fernandes	Team lead | Configuration lead	*K. Fernandes*	05/12/2026
Shaima Nimeri	Design and implementation lead | Requirement lead |Team lead	*Shaima Nimeri*	05/12/2026
Kolya Gavlishin	QA Lead | Design and implementation lead	*Kolya Gavlishin*	5/13/26

Revision history

Version	Author	Date	Change
1.0	**Fatimah, Shaima, Kolya, Kai**	05/11/2026	Initial SPPP draft for planning, scope, QA, risks, management plans, and AI approach.

1. Overview

Motivation:

Bariatric surgery programs require patients to complete a multi-stage evaluation process involving multiple specialists including surgeons, psychologists, dietitians, and endoscopic specialists before receiving surgical clearance. Today this process is managed through phone calls, spreadsheets, paper checklists, and disconnected systems. Patients have no visibility into their own progress, coordinators manually track dozens of patients across dozens of status columns, and program directors have no real-time insight into pipeline health. This fragmentation causes delays, missed appointments, lost documentation, and patient dropout.

Purpose:

To create a web based clinical evaluation management system that digitizes and centralizes the entire bariatric journey evaluation workflow. It gives patients a self service portal to track their progress, gives coordinators a single dashboard to manage all patient statuses and order updates , and gives program directors a real time view of the full patient pipeline. For

simplicity we used a table to mimic patient electronic medical records similar to Epic, which would be connected to the app where the app reflects all updates by the involved care team including psychologists, dietitians, cardiologists, nephrologists, lab-technicians and gastroenterologists. We used the table as the single source of truth. Every update made by the coordinator instantly reflects in both the coordinator portal and the patient portal.

Potential users:

Bariatric patients, beriatric program coordinator, beriatric program director

Proposed software system:

BariatricPath a web based clinical evaluation management system.

Basic functionality of the proposed software system:

- Patient self registration with BMI based specialist recommendation
- Role based login routing each user to their correct screen
- Patient portal with appointment checklist, notifications, and AI chatbot
- Coordinator dashboard with full patient management and column update capability.
- Program director dashboard with sortable filterable patient table
- Master patient table as single source of truth coordinator updates flow to patient portal automatically
- AI powered chatbot answering patient questions based on their live checklist data.
- Audit logging of every status change with a time stamp.

Possible technology stack to be used:

Layer	**Technology**
Frontend	React.js
Backend	Node.js + Express REST API
Database	PostgresSQL+Prisma ORM
Authentication	Firebase Authentication
AI	OpenAI GPT-4 API via Express
Project management	Jira + Github
Container	Docker
Deployment	Digital ocean

2. Related Work

1.Epic MyChart

Epic MyChart is the patient-facing portal for the Epic EHR system used by most major US hospitals. Patients can view appointments, lab results, and messages from their care team.

Differences from BariatricPath:

- MyChart is a general-purpose patient portal for all medical care. BariatricPath is purpose-built specifically for bariatric surgery evaluation workflows.
- MyChart does not include BMI-based specialist recommendation logic, program-specific checklists, or an AI chatbot trained on bariatric program context.
- BariatricPath's coordinator dashboard gives direct column-level update capability mirroring the clinical workflow. MyChart does not expose this kind of structured checklist management to coordinators.

2.**Salesforce Health Cloud**

Salesforce Health Cloud is a care management platform used by healthcare organizations to track patient journeys and coordinate care.

Differences from BariatricPath:

- Salesforce Health Cloud is a general enterprise platform requiring significant customization, licensing costs, and IT resources to implement for a specific program. BariatricPath is pre-configured for the bariatric surgery evaluation workflow out of the box.
- Health Cloud does not include a patient-facing portal with a program-specific checklist or AI chatbot.
- BariatricPath includes automatic BMI-based routing logic and visit-type-specific requirement generation that would require custom development in Salesforce.

3. Proposed High level Requirements

a. Functional Requirements

i. Essential Features (the core features that you definitely need to finish):

Authentication and Role Management

- Patient account creation – As a patient I want to create an account with my email and password so that I can access my patient portal securely.
- Role based navigation – As a user I want the system to route me to the correct dashboard based on my role so that I only see what is relevant to me.
- Role assignment to users – As an administrator, I want to assign roles to users so that each person has the correct level of access
- Staff log out – As a staff member I want to log out of the system so that my account stays secure

Patient Authentication Recommendation:

- BMI calculation – As a patient I want to enter my height and weight so that the system calculates my BMI automatically.
- Specialist recommendations – As a patient I want to receive a specialist recommendation based on my BMI and related medical history so that I am directed to the appropriate provider.

- BMI requirements – As a patient I want to be told I am not eligible if my BMI is below 27 so that I understand why I cannot schedule an appointment.
- Patient confirmation – As a patient I want to see a thank you confirmation after submitting my profile so that I know my coordinator will contact me.

Patient Portal:

- Appointment checklist – As a patient I want to see my appointment checklist with the status of each item so that I know what has been completed and what is still pending.
- Checklist auto update – As a patient I want my checklist to update automatically when the coordinator makes a change so that I always see my most current status.
- Checklist update patient notification – As a patient I want to receive notification when my coordinator updates my checklist time so that I stay informed without having to check manually.
- Patient cleared for next step – As a patient I want my portal to show Surgery Cleared when all requirements are met so that I know I am ready for the next step.
- AI chatbot for patient questions – As a patient I want to ask an AI chatbot questions about my program status so that I can get answers at any time without calling the coordinator.

Coordinator Dashboard:

- Search for patients – As a coordinator I want to search for a patient by name, date of birth or MRN so that I can quickly find who I need to work with.
- Patient list – As a coordinator I want to see a list of all patients with their insurance status and progress so that I can prioritize who needs attention.
- Update insurance – As a coordinator I want to update the insurance status column for any patient so that the patient portal reflects the current insurance situation.
- Update clinical order – As a coordinator I want to update any clinical order column for a patient so that the master table and patient portal stay current.
- Updates auto publish to patient – As a coordinator I want the patient portal to update automatically when I save a column change so that the patient sees the update without me contacting them.
- Notification sent to patient – As a coordinator I want a notification to be sent to the patient whenever I update their checklist so that they are informed of every change.

- View new patient registrations – As a coordinator I want to see new patient registrations so that I know who to call to schedule appointments.

Audit and Compliance

- Timestamp and revision history for updates – As an administrator I want every column update logged with the user timestamp and old and new values so that there is a complete audit trail for every patient record.

ii. Desirable Features (the nice features that you really want to have too):

Program Director Dashboard

- View single patient – As a program director I want to click on a patient and see their full detailed view so that I can drill down into individual cases.
- View all patients – As a program director I want to see all patients across the program in a single table so that I have full visibility into the pipeline.
- Filter patients by specialist – As a program director I want to filter patients by specialist type so that I can see patients grouped by the type of doctor they are seeing.
- Filter patients by stage progress – As a program director I want to sort patients by stage progress so that I can identify who is furthest along and who is just starting.
- Pipeline summary – As a program director I want to see pipeline summary metrics so that I can monitor program efficiency at a glance.

Audit and Compliance

- View patient history – As a coordinator I want to see a patient history log so that I can review what changed and when.

iii. Optional Features (additional cool features that you want to have if there is time):

- Ask AI bot – As a patient, I want to ask the AI bot questions so that I can view my status based on live checklist data.
- Insurance checklist – As a coordinator, I would like to create a checklist so that I can keep track of insurance acceptance
- Patient scheduling – As a coordinator, I would like to view schedules of other coordinators so that I can schedule patients based on current availability.
- Insurance letter generation – As a coordinator, I would like to leverage an AI bot to auto generate insurance letters for patients.
- Email notifications – As a patient, I would like to receive email notifications so that I know when checklist time is updated

b. Nonfunctional Requirements

i. Security requirements

- **Authentication**: all users must authenticate via Firebase before accessing any protected route. JWT tokens are verified on every Express API request.
- **Role based access control:** every API endpoint is protected by role middleware. Patients can only access their own data. Coordinators can access all patient data but cannot access director analytics. Program directors have read-only access to all data.
- **Data privacy:** patients can never see other patients' data. All patient data is stored only in the application's PostgreSQL database. No patient data is sent to third-party services except the OpenAI API for AI chatbot responses. The chatbot only receives the requesting patient's own checklist data.
- **Input validation:** all API inputs are validated server-side using express-validator before being written to the database. Column update requests are validated against a whitelist of allowed column names to prevent injection attacks.
- **Audit trail:** every data modification is logged in the audit_logs table with user ID, patient ID, column changed, old value, new value, and timestamp. Logs are immutable and cannot be deleted through the application.
- **HTTPS**: all production traffic is served over HTTPS. API keys and database credentials are stored as environment variables and never committed to GitHub.
- **Rate limiting AI chatbot endpoints:** are rate limited to 20 requests per 15-minute window per user to prevent abuse and control API costs.

nonfunctional requirements:

- **Compatibility :**
- The application must work on Chrome, Firefox, Safari, and Edge on desktop and mobile browsers.
- Bootstrap 5 provides responsive layout for all screen sizes.
- **Scalability:**
- The PostgreSQL + Prisma stack supports horizontal scaling. Supabase provides managed scaling for the database tier.
- Express API is stateless and can be scaled horizontally behind a load balancer.
- **Reliability:**Target uptime of 99% during business hours.
- Render and Supabase both provide automatic failover and backup.
- GitHub Actions CI/CD pipeline ensures only tested code is deployed to production.

4. Management Plan

a. Objectives and Priorities

- **Priority 1** : Complete all essential features All 21 essential user stories must be implemented, tested, and deployed by the end of Week 6.
- **Priority 2** : Deploy successfully accessible via public URL by Week 6.
- **Priority 3** : Maintain high code quality All pull requests reviewed by at least one teammate. Test coverage above 70% for all Express routes. AI usage fully documented per CS673 requirements.
- Priority 4 : Deliver desirable features AI chatbot, coordinator audit history, and program director detail view completed if essential features are finished ahead of schedule.

b. Risk Management (need to be updated constantly)

The highest priority risks we identified include not having enough time for integration and deployment, scope creep, improper planning, and improper task assignment. We will address these by planning early in each iteration and assigning clear tasks and goals for each team member. We will make sure we focus on our essential requirements first before we work on the desired features. We will also make sure each team member is familiar with the tasks that they are responsible for, and if they are not they will do individual research and their team members will support where needed.

Risk Management Sheet Link:
https://docs.google.com/spreadsheets/d/144Os_XzIdNpH4aGQ5XzWRQmNl6xaI2ImAhfMUd7RhjE/edit?gid=0#gid=0

c. Timeline

Iteration	Functional Requirements(Essential/Disable/Option)	Tasks (Cross requirements tasks)	Estimated/real total person hours
0	setup, project configuration	GitHub repo setup, Jira board, SPPP document, team roles, tech stack decision, PostgreSQL schema design, Firebase project setup, development environment setup	55 / 63.5

1			
2			
3			

5. Configuration Management Plan

a. Tools

Frontend: React.js
Backend: Node.js, Express REST API
Authentication: Firebase Authentication
Database: PostgreSQL + Prisma ORM
Version Control: Git, GitHub
Container: Docker
Project Management: Jira
AI Tools: OpenAI API GPT-4 API via Express
Deployment: Digital ocean
Testing: Jest, React Testing Library, Playwright or Cypress
IDE / Development Tool: Visual Studio Code
CI/CD Tool: GitHub Actions
SAST Tool: ESLint
DAST Tool: OWASP ZAP

b. Code Commit Guideline and Git Branching Strategy

We will use a branching strategy similar to GitHub Flow. The main branch will always contain stable and deployable code from our latest release. There will be a dev branch built off of main where all team members will push changes to. All new features, bug fixes, and experimental changes will be developed in separate self-contained feature branches created from dev. Each branch will focus on a single feature or fix and should remain short-lived (<2 days) to reduce merge conflicts and keep development synchronized. Branches active for more than 2 days should regularly pull updates from dev/main to ensure compatibility with the latest codebase. Once development and testing are complete, changes will be submitted through a pull request. Pull requests will be reviewed by at least one team member before merging into dev. Once a week, when our dev branch is stable we will push changes to the main branch after testing to ensure it is deployable. Commits will follow the class structure of labeling [AI], [HYBRID], [HUMAN] based on level of AI contribution.

c. CI/CD Plan

We plan to use GitHub Actions to support continuous integration throughout the project lifecycle. Whenever code is pushed to a branch or a pull request is created, automated workflows will run to verify that the project builds successfully and passes all tests. All development changes will be integrated through pull requests to ensure code review and validation before merging into the main branch. For deployment, stable versions of the application will be released on GitHub at the end of each sprint cycle.

6. Quality Assurance Plan

a. Metrics

The goal is to define measurable quality expectations and practical review/testing processes from beginning, not coding it finish.

I will break it down into 3 measures:

Fully automated (tools track them for us - minimal manual work)

Metric Name	How It's tracked	Where
Lines of code(LOC)	`cloc` tool runs in CI, outputs a number	GitHub Actions log.
File / class / method count	ESLint or static analysis in CI	CI output
Cyclomatic complexity	ESLint complexity rule per function.	CI output / PR comment
Test coverage	Jest `--coverage` flag	CI output, Codecov badge.
Number of test cases	Jest output	CI log
Test pass rate	CI pass/fail	GitHub Actions dashboard

Semi-automated(tools generate data, we summarize it)

Metric	How It's tracked	Effort
Code review turnaround	`GitHub PR timestamps`	Use a GitHub query - ~15 min/iteration
Stories planned vs completed	GitHub Projects board	Visible on board — ~5 min/iteration
Defect rate (defects/KLOC)	GitHub Issues count / KLOC	Simple division — ~5 min/iteration
Defect density per module	GitHub Issues labeled by area	Light manual tagging — ~10 min/iteration
Number of test cases	Jest output	CI log
Defect resolution time	GitHub Issues open/close timestamps	GitHub query — ~10 min/iteration
Security findings (SAST)	OWASP ZAP scan output	Manual scan once per iteration — ~30 min

Fully manual

Metric Name	How It's tracked	Effort
Person-hours actual vs estimated	Each person logs in their weekly progress report	Already required by course

b. Coding Standard

- JavaScript / TypeScript: use Airbnb style guide via ESLint config, with Prettier for auto-formatting. PRs fail CI if linting fails.
- TypeScript strict mode enabled (strict: true in tsconfig.json) so the compiler catches type issues.
- Naming: camelCase for variables and functions, PascalCase for components and classes, SCREAMING_SNAKE_CASE for constants, kebab-case for file names.
- Functions kept short: target < 30 lines; cyclomatic complexity < 10. Long functions are a refactoring target.
- No console.log in production code paths — use a centralized logger (winston or pino).
- Avoid magic numbers and strings defined as named constants.
- All public API endpoints documented in OpenAPI/Swagger format.
- Every source file begins with the AI-USAGE SUMMARY block specified in the project description (tools used, percent AI contribution, areas, modifications).
- Comments explain why, not what. Self-documenting code is preferred over heavy commenting.

c. Code Review Process

Everyone reviews documents collaboratively (SPPP, SDD, STD). For code, the process is:

1. All code changes flow through GitHub pull requests — direct commits to main are blocked by branch protection.
2. Every PR requires at least one reviewer's approval before merge.
3. Reviewers are typically peers; the Design and Implementation Leader reviews any change touching architecture or shared components; the Security Leader reviews any change touching authentication, authorization, input validation, or document handling; the QA Leader reviews tests.
4. Reviewers use the team's PR Review Checklist (see below). At minimum a reviewer must verify: (a) code matches the user story / acceptance criteria, (b) tests are present and meaningful, (c) coding standard followed, (d) no obvious security or performance regressions, (e) AI-assisted code is properly annotated.
5. Constructive feedback only — comments name the issue and suggest an improvement, not just "this is wrong".
6. The author addresses comments by either fixing the code or replying with the reason it should stay. A reviewer's "approve" is required to merge.
7. After merge, the branch is deleted to keep the repository clean.

PR Review Checklist (template - to be added to .GitHub/pull_request_template.md)

- PR title and description clearly state intent and link the user story or issue number.
- Tests included for new logic; tests pass in CI.
- Linting and formatting pass in CI.
- Code coverage has not dropped below threshold.
- No secrets, keys, .env, or sensitive data committed.
- Input validation present on all new endpoints.
- Authorization checks present on all new authenticated routes.
- AI-assisted code blocks include the required annotation comments.
- Documentation (README, API docs, SPPP/SDD/STD) updated where relevant.
- Commit messages follow Conventional Commits.

d. Testing

Levels and types

Unit testing: Each developer writes unit tests for their own functions and components. Backend uses Jest with Supertest for endpoint testing; frontend uses Jest + React Testing Library. Mocks/stubs used for external services (LLM API, database in some unit tests) so unit tests are fast and deterministic.

Integration testing: QA Leader coordinates integration tests that exercise multi-component flows: e.g., creating a patient – uploading a document – asking the AI a question – seeing the audit log entry. These run against a Dockerized PostgreSQL and a mocked LLM.

System testing: End-to-end tests of major user journeys run before each demo. Started manually in Iter 1, partially automated by Iter 3 (Playwright or Cypress).

Domain testing: Risk-based test cases designed around equivalence classes and boundary conditions for inputs like BMI (boundary at 18.5, 30, 35, 40, 50), insurance fields (valid carriers, blank, malformed), age (under 18 vs adult), and document size (empty PDF, 0 KB, very large).

Security testing: Static (npm audit, CodeQL, ESLint security plugin) in CI on every PR. Dynamic (OWASP ZAP baseline scan) once per iteration. Manual checks for OWASP Top 10 – broken auth, injection, sensitive data exposure.

Acceptance testing: Driven by Given/When/Then acceptance criteria attached to each user story. The customer (instructor/facilitator) validates at the end-of-iteration demo.

Personnel

- Each developer unit-tests their own code before opening a PR.
- QA Leader focuses on integration and system testing strategy, defines testing standards, owns the STD.
- Security Leader runs and reviews SAST + DAST scans, and owns NFR-S compliance.
- Pair testing during iteration demo – every member runs at least one acceptance test live.

Schedule

- Unit tests written alongside code (TDD-leaning where practical).
- Integration tests written within the same iteration as the feature they cover.
- Full regression suite runs in CI on every PR; smoke tests run after every deployment.
- System / acceptance testing in the last 3 days of each iteration.

Objectives

- 100% of essential user stories have at least one automated acceptance-criteria-driven test.
- Backend coverage >= 60% in Iter 1,>= 75% by Iter 3.
- Zero open high/critical bugs at iteration end.
- Zero open high/critical security findings at iteration end.

e. Defect Management

- Tool: GitHub Issues with labels: bug, enhancement, security, AI-issue, blocker, needs-info.

- Severity levels: Critical (system unusable / security breach / data loss), High (key feature broken), Medium (feature impaired but workaround exists), Low (cosmetic / minor).
- Workflow: New – Triaged – In Progress – In Review – Fixed – Verified – Closed.
- Triage cadence: at every team meeting; Critical bugs triaged within 24 hours of report.
- Responsible parties: anyone files an issue; Team Leader or relevant role leader assigns; original developer typically fixes; QA Leader verifies.
- Defect types tracked: functional bugs, UI bugs, security findings, performance regressions, documentation defects, AI hallucinations or incorrect citations.
- Each bug fix PR references the issue number so the link is permanent in the Git history.

7. AI usage Log

Section	Who	AI contribution (%)	Description	Verified by
Section 1 Section 2 Section 3 Section 4 (a, c) 2 references	Shaima Nimeri	30%	Used AI to be able to communicate design ideas and vision clearly and professionally.	Fatimah Hassan, Kolya Gavlishin, Kai Fernandes
Section 4 (b)	Kai Fernandes	0%		Kolya Gavlishin Shaima Nimeri
Section 5 – Configuration Management Plan	Kai Fernandes	<10%	AI assisted with rewording summary and suggested which of my provided branching strategies would be best for git	Fatimah Hassan, Kolya Gavlishin Shaima Nimeri
Section 6 – QA Plan 7 References Glossary table	Fatimah Hassan	30%	AI assisted with metric definitions and PR checklist structure, metrics and tests adapted to the project ,	Kolya Gavlishin Shaima Nimeri, Kai Fernandes

Section	Who	AI contribution (%)	Description	Verified by
			checklist reviewed and refined.	
Section 7 – AI Usage Log	All members	0%	Self-referential, documented at the end of iteration 0.	
Glossary	Fatimah Hassan	< 10%	AI suggested entries; team verified definitions.	Kolya Gavlishin, Kai Fernandes

8. Project Links

Github link: https://github.com/BUMETCS673/cs673olsum26project-cs673olsum26team1
Project management tool link (e.g. Jira): https://cs673-sum26-team1.atlassian.net/jira/software/projects/SCRUM/boards/1?sprintStarted=true

9. References

- Microsoft Security Development Lifecycle (SDL).
- OWASP SAMM — Software Assurance Maturity Model.
- LangChain Documentation — https://docs.langchain.com
- LangChain.js GitHub — https://github.com/langchain-ai/langchainjs
- ChromaDB Documentation — https://docs.trychroma.com
- Conventional Commits — https://www.conventionalcommits.org
- CS673 Project Description and SPPP template, BU MET, Summer 2026
- Github Branching strategies — https://dev.to/karmpatel/git-branching-strategies-a-comprehensive-guide-24kh
- https://www.salesforce.com
- https://www.mychart.org

10. Glossary

Term	Definition

AI	Artificial Intelligence. This project primarily refers to LLMs (Large Language Models) used for retrieval-augmented Q&A.
BMI	Body Mass Index — clinical metric used as part of bariatric surgery eligibility.
CI / CD	Continuous Integration / Continuous Deployment (or Delivery). Automated build, test, and deploy on every change.
DAST	Dynamic Application Security Testing – security testing on a running application (e.g., OWASP ZAP).
DRY	Don't Repeat Yourself — design principle.
INVEST	Independent, Negotiable, Valuable, Estimable, Small, Testable – user story quality criteria.
JWT	JSON Web Token – used for stateless authentication.
KISS	Keep It Simple, Stupid – design principle.
LLM	Large Language Model – e.g., GPT-4, Claude.
MRN	Medical Review Nurse
MVC	Model-View-Controller – architectural pattern.
MVP	Minimum Viable Product – smallest version of a product that delivers value.
RAG	Retrieval-Augmented Generation – pattern where retrieved documents ground an LLM's answer.
RBAC	Role-Based Access Control.
SAST	Static Application Security Testing –analyzing source code for security issues.
SDD	Software Design Document.
SOLID	Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion.
SOP	Standard Operating Procedure.
SPPP	Software Project Proposal and Planning – this document.
STD	Software Testing Document.
TDD / BDD	Test-Driven Development / Behavior-Driven Development.

XP	Extreme Programming – agile methodology.